



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

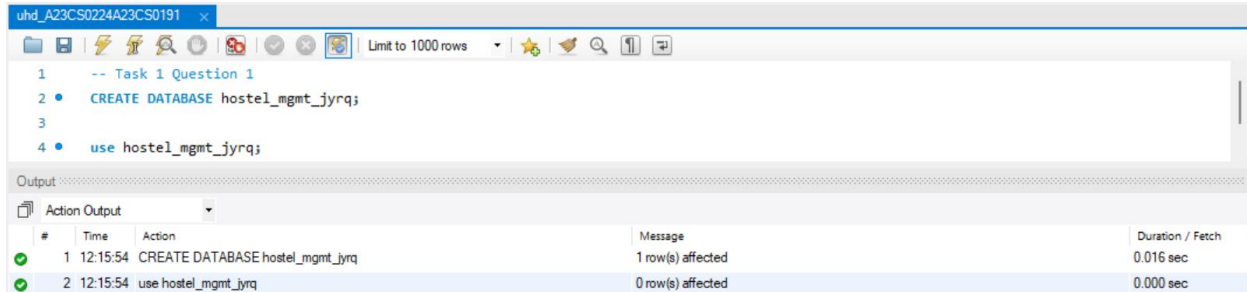
Name	:	Goe Jie Ying
Matric Id	:	A23CS0224
Section	:	2

Name	:	Teh Ru Qian
Matric Id	:	A23CS0191
Section	:	2

Task 1 - Database Creation and Integrity Constraints

Question 1

This command creates a new database *hostel_mgmt_jyrq* and sets it as the current working database.



The screenshot shows a database client window with a tab labeled 'uhd_A23CS0224A23CS0191'. The SQL editor contains the following commands:

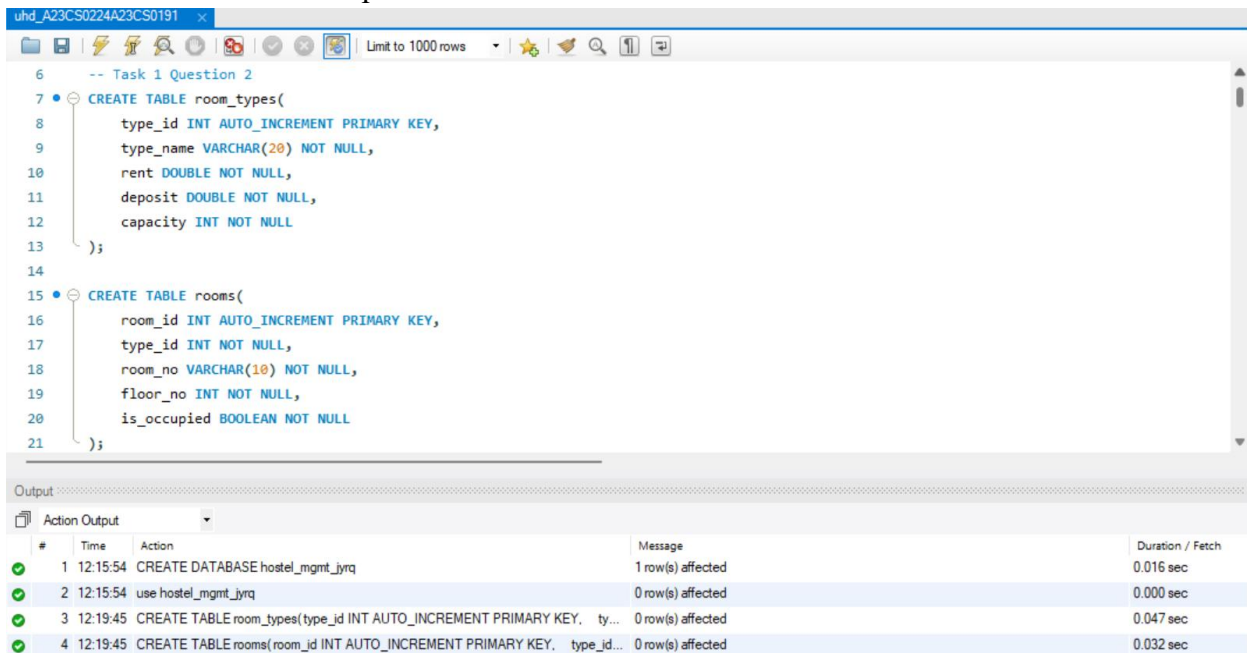
```
1 -- Task 1 Question 1
2 • CREATE DATABASE hostel_mgmt_jyrq;
3
4 • use hostel_mgmt_jyrq;
```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
✓ 1	12:15:54	CREATE DATABASE hostel_mgmt_jyrq	1 row(s) affected	0.016 sec
✓ 2	12:15:54	use hostel_mgmt_jyrq	0 row(s) affected	0.000 sec

Question 2

Tables are created to store specific details.



The screenshot shows a database client window with a tab labeled 'uhd_A23CS0224A23CS0191'. The SQL editor contains the following commands:

```
6 -- Task 1 Question 2
7 • CREATE TABLE room_types(
8     type_id INT AUTO_INCREMENT PRIMARY KEY,
9     type_name VARCHAR(20) NOT NULL,
10    rent DOUBLE NOT NULL,
11    deposit DOUBLE NOT NULL,
12    capacity INT NOT NULL
13 );
14
15 • CREATE TABLE rooms(
16    room_id INT AUTO_INCREMENT PRIMARY KEY,
17    type_id INT NOT NULL,
18    room_no VARCHAR(10) NOT NULL,
19    floor_no INT NOT NULL,
20    is_occupied BOOLEAN NOT NULL
21 );
```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
✓ 1	12:15:54	CREATE DATABASE hostel_mgmt_jyrq	1 row(s) affected	0.016 sec
✓ 2	12:15:54	use hostel_mgmt_jyrq	0 row(s) affected	0.000 sec
✓ 3	12:19:45	CREATE TABLE room_types(type_id INT AUTO_INCREMENT PRIMARY KEY, ty...	0 row(s) affected	0.047 sec
✓ 4	12:19:45	CREATE TABLE rooms(room_id INT AUTO_INCREMENT PRIMARY KEY, type_id...	0 row(s) affected	0.032 sec

Table *room_types* and *rooms* store room details.

uhd_A23CS0224A23CS0191

```

23 CREATE TABLE students(
24     student_id INT AUTO_INCREMENT PRIMARY KEY,
25     room_id INT,
26     fname VARCHAR(30) NOT NULL,
27     lname VARCHAR(30) NOT NULL,
28     status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL,
29     checkin_date DATE,
30     email VARCHAR(255) NOT NULL
31 );
32

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	12:15:54	CREATE DATABASE hostel_mgmt_jyrg	1 row(s) affected	0.016 sec
✓ 2	12:15:54	use hostel_mgmt_jyrg	0 row(s) affected	0.000 sec
✓ 3	12:19:45	CREATE TABLE room_types(type_id INT AUTO_INCREMENT PRIMARY KEY, typ...	0 row(s) affected	0.047 sec
✓ 4	12:19:45	CREATE TABLE rooms(room_id INT AUTO_INCREMENT PRIMARY KEY, type_id ...	0 row(s) affected	0.032 sec
✓ 5	12:24:24	CREATE TABLE students(student_id INT AUTO_INCREMENT PRIMARY KEY, ro...	0 row(s) affected	0.031 sec

Table *students* store student details.

uhd_A23CS0224A23CS0191

```

32
33 CREATE TABLE maintenance(
34     maint_id INT AUTO_INCREMENT PRIMARY KEY,
35     room_id INT NOT NULL,
36     issue_desc VARCHAR(255) NOT NULL,
37     severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,
38     status ENUM('OPEN', 'RESOLVED') NOT NULL,
39     reported_on DATE NOT NULL,
40     resolved_on DATE
41 );
42

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	12:15:54	CREATE DATABASE hostel_mgmt_jyrg	1 row(s) affected	0.016 sec
✓ 2	12:15:54	use hostel_mgmt_jyrg	0 row(s) affected	0.000 sec
✓ 3	12:19:45	CREATE TABLE room_types(type_id INT AUTO_INCREMENT PRIMARY KEY, typ...	0 row(s) affected	0.047 sec
✓ 4	12:19:45	CREATE TABLE rooms(room_id INT AUTO_INCREMENT PRIMARY KEY, type_id ...	0 row(s) affected	0.032 sec
✓ 5	12:24:24	CREATE TABLE students(student_id INT AUTO_INCREMENT PRIMARY KEY, ro...	0 row(s) affected	0.031 sec
✓ 6	12:24:24	CREATE TABLE maintenance(maint_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.032 sec

Table *maintenance* store maintenance details.

The screenshot shows a database management tool interface. The top pane contains SQL code for creating a table named `payments`. The code is as follows:

```

41 );
42
43 CREATE TABLE payments(
44     payment_id INT AUTO_INCREMENT PRIMARY KEY,
45     student_id INT NOT NULL,
46     amount DOUBLE NOT NULL,
47     paid_on DATE NOT NULL,
48     method ENUM('CASH', 'FPX', 'CARD', 'TNG') NOT NULL,
49     note VARCHAR(255)
50 );
51

```

The bottom pane shows the 'Output' window with a table of execution logs. The table has four columns: #, Time, Action, and Message. The logs show the successful execution of several SQL statements, including the creation of the `payments` table.

#	Time	Action	Message	Duration / Fetch
1	12:15:54	CREATE DATABASE hostel_mgmt_jyq	1 row(s) affected	0.016 sec
2	12:15:54	use hostel_mgmt_jyq	0 row(s) affected	0.000 sec
3	12:19:45	CREATE TABLE room_types(type_id INT AUTO_INCREMENT PRIMARY KEY, typ...	0 row(s) affected	0.047 sec
4	12:19:45	CREATE TABLE rooms(room_id INT AUTO_INCREMENT PRIMARY KEY, type_id ...	0 row(s) affected	0.032 sec
5	12:24:24	CREATE TABLE students(student_id INT AUTO_INCREMENT PRIMARY KEY, ro...	0 row(s) affected	0.031 sec
6	12:24:24	CREATE TABLE maintenance(maint_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.032 sec
7	12:27:37	CREATE TABLE payments(payment_id INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.047 sec

Table *payments* store payment details.

Question 3

Unique constraints are applied to ensure that room numbers and room type names are not duplicated. Foreign key constraints are applied to link tables logically.

1. Unique Constraints:

- *Room_types.type_name*: Ensures that each room type is unique.
- *Rooms.room_no*: Ensures that each room number is unique.

2. Foreign Key Constraints:

- *rooms.type_id* → *room_types.type_id*: Each room must belong to a valid room type.
- *students.room_id* → *rooms.room_id*: Each student is assigned to an existing room.
- *maintenance.room_id* → *rooms.room_id*: Maintenance records are linked to existing rooms.
- *payments.student_id* → *students.student_id*: Payments must be associated with valid students.

uhd_A23CS0224A23CS0191

```

52 -- Task 1 Question 3
53 ALTER TABLE room_types
54 ADD CONSTRAINT uq_type_name UNIQUE (type_name);
55
56 ALTER TABLE rooms
57 ADD CONSTRAINT uq_room_no UNIQUE (room_no),
58 ADD CONSTRAINT fk_rooms_type FOREIGN KEY (type_id) REFERENCES room_types(type_id);
59
60 ALTER TABLE students
61 ADD CONSTRAINT fk_students_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);
62
63 ALTER TABLE maintenance
64 ADD CONSTRAINT fk_maintenance_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);
65
66 ALTER TABLE payments
67 ADD CONSTRAINT fk_payments_student FOREIGN KEY (student_id) REFERENCES students(student_id);

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 10	12:40:40	ALTER TABLE students ADD CONSTRAINT fk_students_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.094 sec
✓ 11	12:40:40	ALTER TABLE maintenance ADD CONSTRAINT fk_maintenance_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.141 sec
✓ 12	12:40:40	ALTER TABLE payments ADD CONSTRAINT fk_payments_student FOREIGN KEY (student_id) REFERENCES students(student_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec

Question 4

Data was inserted into all tables in the correct foreign-key order to ensure valid and consistent records for the hostel system.

uhd_A23CS0224A23CS0191

```

68
69 -- Task 1 Question 4
70 INSERT INTO room_types (type_name, rent, deposit, capacity)
71 VALUES
72 ('Economy', 300.00, 150.00, 1),
73 ('Single', 500.00, 200.00, 1),
74 ('Standard', 600.00, 250.00, 2),
75 ('Double', 800.00, 300.00, 2),
76 ('Executive', 900.00, 300.00, 2),
77 ('Triple', 1000.00, 350.00, 3),
78 ('Premium', 1200.00, 400.00, 3),
79 ('Family', 1500.00, 500.00, 4),
80 ('Suite', 1800.00, 600.00, 4),
81 ('Deluxe', 2500.00, 800.00, 6);
82

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 10	12:40:40	ALTER TABLE students ADD CONSTRAINT fk_students_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.094 sec
✓ 11	12:40:40	ALTER TABLE maintenance ADD CONSTRAINT fk_maintenance_room FOREIGN KEY (room_id) REFERENCES rooms(room_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.141 sec
✓ 12	12:40:40	ALTER TABLE payments ADD CONSTRAINT fk_payments_student FOREIGN KEY (student_id) REFERENCES students(student_id);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
✓ 13	12:42:02	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Economy', 300.00, 150.00, 1), ('Single', 500.00, 200.00, 1), ('Standard', 600.00, 250.00, 2), ('Double', 800.00, 300.00, 2), ('Executive', 900.00, 300.00, 2), ('Triple', 1000.00, 350.00, 3), ('Premium', 1200.00, 400.00, 3), ('Family', 1500.00, 500.00, 4), ('Suite', 1800.00, 600.00, 4), ('Deluxe', 2500.00, 800.00, 6);	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec

uhd_A23CS0224A23CS0191

Limit to 1000 rows

82

83 • INSERT INTO rooms (type_id, room_no, floor_no, is_occupied)

84 VALUES

85 (1, 'A101', 1, TRUE),

86 (2, 'A201', 1, FALSE),

87 (3, 'A301', 2, FALSE),

88 (4, 'A401', 3, TRUE),

89 (5, 'B101', 1, FALSE),

90 (6, 'B201', 2, FALSE),

91 (7, 'B301', 3, TRUE),

92 (8, 'C101', 1, FALSE),

93 (9, 'C201', 2, TRUE),

94 (10, 'C301', 3, FALSE);

95

96 • INSERT INTO students (room_id, fname, lname, status, checkin_date, email)

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 11	12:40:40	ALTER TABLE maintenance ADD CONSTRAINT fk_maintenance_room FOREIGN KE...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.141 sec
✓ 12	12:40:40	ALTER TABLE payments ADD CONSTRAINT fk_payments_student FOREIGN KEY (st...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
✓ 13	12:42:02	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Economy', 30...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
✓ 14	14:59:53	INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES (1, 'A101', 1, T...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.141 sec

uhd_A23CS0224A23CS0191

Limit to 1000 rows

95

96 • INSERT INTO students (room_id, fname, lname, status, checkin_date, email)

97 VALUES

98 (1, 'Ahmad', 'Zaki', 'ACTIVE', '2025-11-01', 'ahmad@email.com'),

99 (2, 'Bruce', 'Lee', 'NON_ACTIVE', '2025-10-22', 'bruce@email.com'),

100 (3, 'Camily', 'Hong', 'NON_ACTIVE', '2024-01-02', 'camily@email.com'),

101 (4, 'Darren', 'Tan', 'ACTIVE', '2025-11-08', 'darren@email.com'),

102 (5, 'Elisa', 'Alya', 'NON_ACTIVE', '2025-09-10', 'elisa@email.com'),

103 (6, 'Farah', 'Safiya', 'NON_ACTIVE', '2025-10-02', 'farah@email.com'),

104 (7, 'Gamuda', 'Yusof', 'ACTIVE', '2025-11-07', 'gamu@email.com'),

105 (8, 'Hello', 'Kitty', 'NON_ACTIVE', '2025-08-15', 'hello@email.com'),

106 (9, 'Isaac', 'Lau', 'ACTIVE', '2025-11-08', 'isaac@email.com'),

107 (10, 'Jolin', 'Tsai', 'NON_ACTIVE', '2025-07-20', 'jolin@email.com');

108

109 • INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 12	12:40:40	ALTER TABLE payments ADD CONSTRAINT fk_payments_student FOREIGN KEY (st...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
✓ 13	12:42:02	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Economy', 30...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
✓ 14	14:59:53	INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES (1, 'A101', 1, T...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.141 sec
✓ 15	15:00:27	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec

uhd_A23CS0224A23CS0191

Limit to 1000 rows

```

108
109 • INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
110 VALUES
111 (1, 'AC not working', 'MEDIUM', 'OPEN', '2025-11-05', NULL),
112 (2, 'Plug problem', 'LOW', 'RESOLVED', '2025-10-22', '2025-10-23'),
113 (3, 'Fan broken', 'HIGH', 'RESOLVED', '2024-01-02', '2024-01-02'),
114 (4, 'Heater not working', 'MEDIUM', 'OPEN', '2025-11-09', NULL),
115 (5, 'Light not bright', 'LOW', 'RESOLVED', '2025-09-12', '2025-09-14'),
116 (6, 'Water leakage', 'HIGH', 'RESOLVED', '2025-10-03', '2025-10-03'),
117 (7, 'Door handle loose', 'LOW', 'OPEN', '2025-11-07', NULL),
118 (8, 'Window cracked', 'MEDIUM', 'RESOLVED', '2025-08-15', '2025-08-16'),
119 (9, 'Aircon dripping', 'MEDIUM', 'OPEN', '2025-11-08', NULL),
120 (10, 'Curtain stain', 'LOW', 'OPEN', '2025-07-20', '2025-07-23');
121
122

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 13	12:42:02	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Economy', 30...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
✓ 14	14:59:53	INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES (1, 'A101', 1, T...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.141 sec
✓ 15	15:00:27	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
✓ 16	15:01:09	INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolve...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec

uhd_A23CS0224A23CS0191

Limit to 1000 rows

```

121
122
123 • INSERT INTO payments (student_id, amount, paid_on, method, note)
124 VALUES
125 (1, 500.00, '2025-11-01', 'CASH', 'Single'),
126 (2, 800.00, '2025-10-22', 'CARD', 'Double Rent'),
127 (3, 600.00, '2024-01-05', 'FPX', 'Standard Room Rent'),
128 (4, 1500.00, '2025-11-08', 'TNG', 'Family'),
129 (5, 900.00, '2025-09-10', 'FPX', NULL),
130 (6, 1000.00, '2025-10-05', 'CARD', 'Triple'),
131 (7, 1200.00, '2025-11-07', 'CASH', 'Premium'),
132 (8, 1500.00, '2025-08-20', 'TNG', 'Family'),
133 (9, 1800.00, '2025-11-08', 'FPX', 'Suite '),
134 (10, 2500.00, '2025-11-09', 'CARD', 'Deluxe');
135

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 14	14:59:53	INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES (1, 'A101', 1, T...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.141 sec
✓ 15	15:00:27	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
✓ 16	15:01:09	INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolve...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
✓ 17	15:01:41	INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1, 500....	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec

Question 5

A unique constraint was added to *students.email* to prevent duplicates and a temporary table test was created and then dropped to show how temporary or unused tables can be safely removed if they already exist.


```

136 -- Task 1 Question 5
137 • ALTER TABLE students
138 ADD CONSTRAINT uq_email UNIQUE (email);
139
140 • CREATE TABLE test (
141     test_id INT AUTO_INCREMENT PRIMARY KEY,
142     note VARCHAR(100)
143 );
144
145 • DROP TABLE IF EXISTS test;

```

Output

#	Time	Action	Message	Duration / Fetch
✓ 18	15:05:09	ALTER TABLE students ADD CONSTRAINT uq_email UNIQUE (email)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
✓ 19	15:05:10	CREATE TABLE test (test_id INT AUTO_INCREMENT PRIMARY KEY, note VA...	0 row(s) affected	0.031 sec
✓ 20	15:05:10	DROP TABLE IF EXISTS test	0 row(s) affected	0.016 sec

Task 2 - Data Manipulation and Filtering

Question 1

The insertion of data was already completed in **Task 1 Question 4**.

Question 2

The `SET SQL_SAFE_UPDATES = 0;` command is used to temporarily disable MySQL's safety feature that prevents updates or deletes without a `WHERE` clause or key condition. This allows the `UPDATE` statement to mark rooms as occupied based on active students and the `DELETE` statement to remove old resolved maintenance records. After the operations, `SQL_SAFE_UPDATES` is turned back on for safety.

```

147 -- Task 2 Question 2
148 • SET SQL_SAFE_UPDATES = 0;
149
150 • UPDATE rooms r
151 LEFT JOIN students s ON r.room_id = s.room_id
152 SET r.is_occupied =
153     CASE
154         WHEN s.status = 'ACTIVE' THEN TRUE
155         ELSE FALSE
156     END;
157
158 • DELETE FROM maintenance
159 WHERE status = 'RESOLVED' AND reported_on < (CURDATE() - INTERVAL 60 DAY);
160
161 • SET SQL_SAFE_UPDATES = 1;

```

Output

#	Time	Action	Message	Duration / Fetch
✓ 22	15:33:39	UPDATE rooms r LEFT JOIN students s ON r.room_id = s.room_id SET r.is_occupied = ...	0 row(s) affected Rows matched: 10 Changed: 0 Warnings: 0	0.047 sec
✓ 23	15:33:39	DELETE FROM maintenance WHERE status = 'RESOLVED' AND reported_on < (CUR...	2 row(s) affected	0.015 sec
✓ 24	15:33:39	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec

Question 3

This query retrieves all room types with rent values between RM400 and RM800, filtering the dataset to show only mid-range rooms.

The screenshot shows a database query interface with a SQL query editor and a result grid. The query is as follows:

```
162
163 -- Task 2 Question 3
164 • SELECT *
165 FROM room_types
166 WHERE rent BETWEEN 400 AND 800;
167
```

The result grid displays the following data:

type_id	type_name	rent	deposit	capacity
2	Single	500	200	1
3	Standard	600	250	2
4	Double	800	300	2
NULL	NULL	NULL	NULL	NULL

This query retrieves all students whose first name starts with the letter “A”.

The screenshot shows a database query interface with a SQL query editor and a result grid. The query is as follows:

```
167
168 • SELECT *
169 FROM students
170 WHERE fname LIKE 'A%';
171
```

The result grid displays the following data:

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Ahmad	Zaki	ACTIVE	2025-11-01	ahmad@email.com
NULL	NULL	NULL	NULL	NULL	NULL	NULL

This query retrieves all payment records made using either the FPX or CARD payment methods.

The screenshot shows a database query interface with a SQL query editor and a result grid. The query is as follows:

```
172 • SELECT *
173 FROM payments
174 WHERE method IN ('FPX', 'CARD');
175
```

The result grid displays the following data:

payment_id	student_id	amount	paid_on	method	note
2	2	800	2025-10-22	CARD	Double Rent
3	3	600	2024-01-05	FPX	Standard Room Rent
5	5	900	2025-09-10	FPX	NULL
6	6	1000	2025-10-05	CARD	Triple
9	9	1800	2025-11-08	FPX	Suite
10	10	2500	2025-11-09	CARD	Deluxe
NULL	NULL	NULL	NULL	NULL	NULL

This query retrieves room types that either have a rent of at least RM1000 and a deposit of at least RM500, or a capacity of 3 or more.

```
176 • SELECT *
177 FROM room_types
178 WHERE (rent >= 1000 AND deposit >= 500) OR capacity >=3;
179
```

	type_id	type_name	rent	deposit	capacity
▶	6	Triple	1000	350	3
	7	Premium	1200	400	3
	8	Family	1500	500	4
	9	Suite	1800	600	4
	10	Deluxe	2500	800	6
*	NULL	NULL	NULL	NULL	NULL

Question 4

This query counts the total number of students in the students table.

```
uhd_A23CS0224A23CS0191 x
179
180 -- Task 2 Question 4
181 -- Find total number of students
182 • SELECT COUNT(student_id) AS num_of_stud
183 FROM students;
184
```

	num_of_stud
▶	10

This query calculates the average rent across all room types.

```
185 -- Find the average rental fee for all room types
186 • SELECT AVG(rent) AS average_rent
187 FROM room_types;
188
```

	average_rent
▶	1110

This query combines each student's first and last name into a full name and calculates the total number of characters in the combined name.

```
189 -- Combine the first name and last name, then calculate the length
190 • SELECT
191     CONCAT(fname, ' ', lname) AS full_name,
192     LENGTH(CONCAT(fname, lname)) AS name_length
193 FROM students;
```

full_name	name_length
Ahmad Zaki	9
Bruce Lee	8
Camly Hong	10
Darren Tan	9
Elisa Alya	9
Farah Safiya	11
Gamuda Yusof	11
Hello Kitty	10
Isaac Lau	8
Jolin Tsai	9

Task 3 – Reporting and Aggregation

Question 1

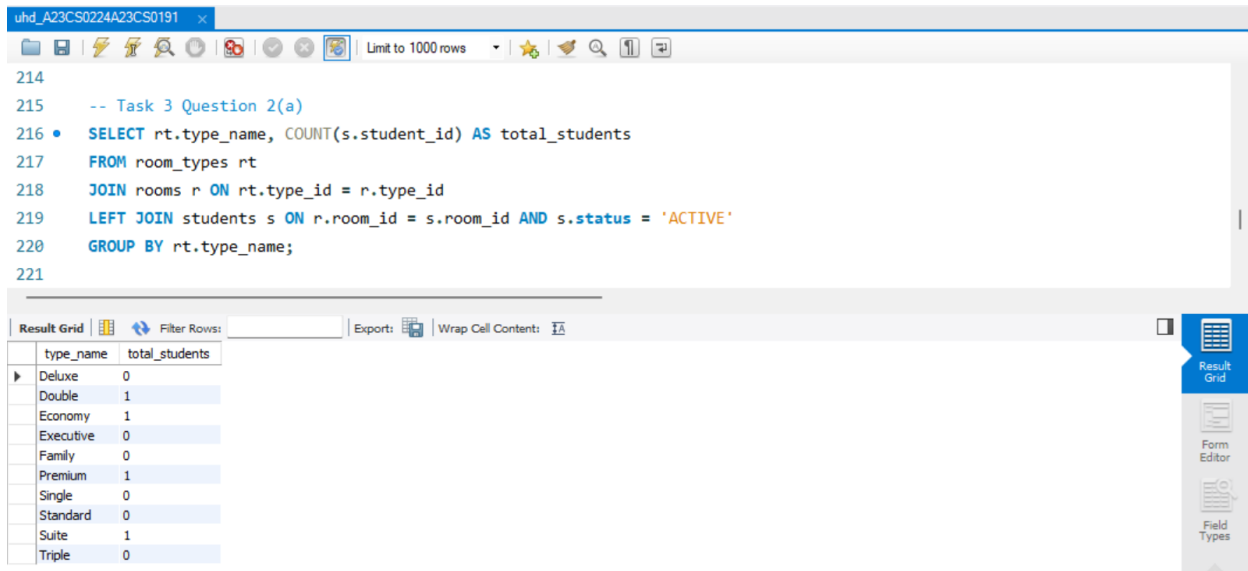
This code creates a view *v_room_status* that shows each room's number, type, rent, floor, capacity, number of active students, open maintenance issues, and whether the room is vacant. It joins the rooms, room_types, students, and maintenance tables, counts active students and open issues, and marks a room as vacant if it has no active students.

```
195 -- Task 3 Question 1
196 • CREATE OR REPLACE VIEW v_room_status AS
197     SELECT
198         r.room_no,
199         rt.type_name,
200         rt.rent,
201         r.floor_no,
202         rt.capacity,
203         COUNT(s.student_id) AS n_occupants,
204         SUM(CASE WHEN m.status = 'OPEN' THEN 1 ELSE 0 END) AS pending_issues,
205         CASE WHEN COUNT(s.student_id) = 0 THEN 1 ELSE 0 END AS is_vacant
206     FROM rooms r
207     JOIN room_types rt ON r.type_id = rt.type_id
208     LEFT JOIN students s ON r.room_id = s.room_id AND s.status = 'ACTIVE'
209     LEFT JOIN maintenance m ON r.room_id = m.room_id
210     GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;
211
212 -- Query the view
213 • SELECT * FROM v_room_status;
```

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Economy	300	1	1	1	1	0
A201	Single	500	1	1	0	0	1
A301	Standard	600	2	2	0	0	1
A401	Double	800	3	2	1	1	0
B101	Executive	900	1	2	0	0	1
B201	Triple	1000	2	3	0	0	1
B301	Premium	1200	3	3	1	1	0
C101	Family	1500	1	4	0	0	1
C201	Suite	1800	2	4	1	1	0
C301	Deluxe	2500	3	6	0	1	1

Question 2

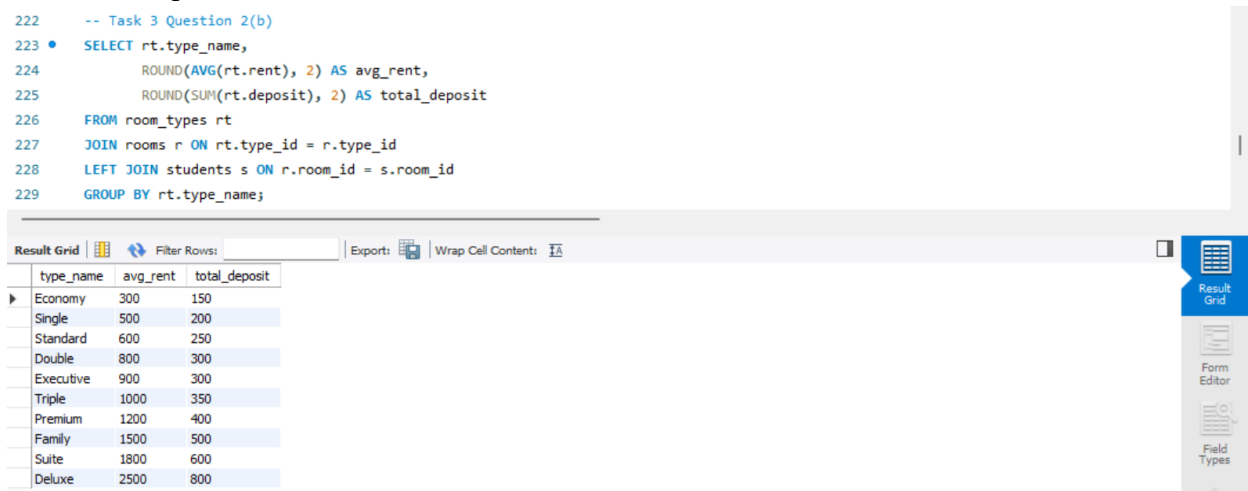
This query counts the number of active students in each room type.



```
214
215 -- Task 3 Question 2(a)
216 • SELECT rt.type_name, COUNT(s.student_id) AS total_students
217 FROM room_types rt
218 JOIN rooms r ON rt.type_id = r.type_id
219 LEFT JOIN students s ON r.room_id = s.room_id AND s.status = 'ACTIVE'
220 GROUP BY rt.type_name;
221
```

type_name	total_students
Deluxe	0
Double	1
Economy	1
Executive	0
Family	0
Premium	1
Single	0
Standard	0
Suite	1
Triple	0

This query calculates the average rent and total deposit for each room type, rounding the values to two decimal places.



```
222 -- Task 3 Question 2(b)
223 • SELECT rt.type_name,
224         ROUND(AVG(rt.rent), 2) AS avg_rent,
225         ROUND(SUM(rt.deposit), 2) AS total_deposit
226 FROM room_types rt
227 JOIN rooms r ON rt.type_id = r.type_id
228 LEFT JOIN students s ON r.room_id = s.room_id
229 GROUP BY rt.type_name;
```

type_name	avg_rent	total_deposit
Economy	300	150
Single	500	200
Standard	600	250
Double	800	300
Executive	900	300
Triple	1000	350
Premium	1200	400
Family	1500	500
Suite	1800	600
Deluxe	2500	800

This query shows the total payment amount for each month and year, rounded to two decimal places.

uhd_A23CS0224A23CS0191

Limit to 1000 rows

```

231
232 -- Task 3 Question 2(c)
233 • SELECT
234     YEAR(p.paid_on) AS year,
235     MONTH(p.paid_on) AS month,
236     ROUND(SUM(p.amount),2) AS total_payment
237 FROM payments p
238 GROUP BY YEAR(p.paid_on), MONTH(p.paid_on)
239 ORDER BY year, month;
240

```

Result Grid

	year	month	total_payment
▶	2024	1	600
	2025	8	1500
	2025	9	900
	2025	10	1800
	2025	11	7500

Result Grid
Form Editor

This query lists floors that have more than two open maintenance issues.

uhd_A23CS0224A23CS0191

Limit to 1000 rows

```

240
241 -- Task 3 Question2(d)
242 • SELECT
243     r.floor_no, COUNT(m.maint_id) AS open_issues
244 FROM maintenance m
245 JOIN rooms r ON m.room_id = r.room_id
246 WHERE m.status = 'OPEN'
247 GROUP BY r.floor_no
248 HAVING COUNT(m.maint_id) > 2;
249

```

Result Grid

	floor_no	open_issues
▶	3	3

Result Grid

Question 3

This query joins multiple tables to show each room's details such as room label, rent (rounded), rent category (LOW, MEDIUM, HIGH), number of active occupants, open maintenance issues, vacancy status and total deposit. It also applies 'CONCAT', 'UPPER', 'LOWER', 'ROUND' and 'CASE' functions. This helps summary room information by combining and formatting data using SQL functions to help in reporting.

uhd_A23CS0224A23CS0191

Limit to 1000 rows

```

250 -- Task 3 Question 3
251 SELECT
252     -- Use CONCAT to combine type + floor
253     CONCAT(rt.type_name, ' - Floor ', r.floor_no) AS room_label,
254     r.room_no,
255     -- Uppercase type
256     UPPER(rt.type_name) AS type_upper,
257     -- Lowercase type
258     LOWER(rt.type_name) AS type_lower,
259     -- Round rent
260     ROUND(rt.rent, 2) AS rent,
261     -- Rent category
262     CASE
263         WHEN rt.rent < 500 THEN 'LOW'
264         WHEN rt.rent BETWEEN 500 AND 1000 THEN 'MEDIUM'
265         ELSE 'HIGH'
266     END AS rent_category,
267     -- Number of active students
268     COUNT(s.student_id) AS n_occupants,
269     -- Open maintenance
270     SUM(CASE WHEN m.status = 'OPEN' THEN 1 ELSE 0 END) AS pending_issues,
271     -- Vacant?
272     CASE WHEN COUNT(s.student_id) = 0 THEN 1 ELSE 0 END AS is_vacant,
273     -- Total deposits
274     ROUND(SUM(rt.deposit), 2) AS total_deposit
275 FROM rooms r
276 JOIN room_types rt ON r.type_id = rt.type_id
277 LEFT JOIN students s ON r.room_id = s.room_id AND s.status = 'ACTIVE'
278 LEFT JOIN maintenance m ON r.room_id = m.room_id
279 GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no
280 ORDER BY rt.type_name, r.floor_no;

```

Result Grid

room_label	room_no	type_upper	type_lower	rent	rent_category	n_occupants	pending_issues	is_vacant	total_deposit
Deluxe - Floor 3	C301	DELUXE	deluxe	2500	HIGH	0	1	1	800
Double - Floor 3	A401	DOUBLE	double	800	MEDIUM	1	1	0	300
Economy - Floor 1	A101	ECONOMY	economy	300	LOW	1	1	0	150
Executive - Floor 1	B101	EXECUTIVE	executive	900	MEDIUM	0	0	1	300
Family - Floor 1	C101	FAMILY	family	1500	HIGH	0	0	1	500
Premium - Floor 3	B301	PREMIUM	premium	1200	HIGH	1	1	0	400
Single - Floor 1	A201	SINGLE	single	500	MEDIUM	0	0	1	200
Standard - Floor 2	A301	STANDARD	standard	600	MEDIUM	0	0	1	250
Suite - Floor 2	C201	SUITE	suite	1800	HIGH	1	1	0	600
Triple - Floor 2	B201	TRIPLE	triple	1000	MEDIUM	0	0	1	350

Result Grid
Form Editor
Field Types

Summary Table

Table Name	Description	Number of Records
room_types	Stores room type details	10
rooms	Stores room details	10
students	Stores student details	10
maintenance	Stores maintenance issues	10
payments	Stores payment transactions	10

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥ 10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more).	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			